Understanding the Transformer Block

From Residual Streams and Attention to MLPs, Normalization, and Modern LLM Blocks

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 09 - Understanding AI Series

- ORIENTATION

What this handbook will make clear

A Transformer block looks simple in diagrams: normalization, attention, a feed-forward network, and two residual additions. The difficulty is that modern language models have changed almost every detail around that skeleton. Some normalize before a sublayer, some after it, some do both. The feed-forward branch may use ReLU, GELU, SwiGLU, GeGLU, or a sparse MoE. Attention may use MHA, GQA, MQA, MLA, RoPE, QK-Norm, local windows, or a KV cache during decoding.

This handbook separates the stable structure from architecture-specific choices. By the end, you should be able to explain:

- what information flows through the residual stream and why every sublayer must return to the model width;
- how Post-LN and Pre-LN differ algebraically, not just diagrammatically;
- what LayerNorm and RMSNorm actually compute;
- where attention begins and ends inside a block, including RoPE, GQA, and the output projection;
- why the MLP is position-wise even though the full block is not;
- how SwiGLU changes the feed-forward computation and parameter count;
- how MoE replaces the dense MLP branch without changing the surrounding residual topology;
- what changes between training, prompt prefill, and incremental decoding;
- which details are universal enough to reason about and which must be read from a model configuration.

Prerequisites

Basic comfort with tensors, matrix multiplication, attention, and neural-network layers is enough. The earlier handbooks on Attention, RoPE, KV Cache, MHA/MQA/GQA/MLA, MoE, and Embeddings are useful context, but the block is rebuilt here from first principles. **Scope**

This is a handbook about the computation inside one Transformer block and how blocks are stacked. It touches kernels, distributed execution, activation checkpointing, and serving only where they clarify block behavior. Those systems topics deserve separate treatment. **Notation used throughout**

B = batch size; T = sequence length; d = model/residual width; H_q = query-head count; H_{kv} = key/value-head count; d_h = head dimension; m = feed-forward intermediate width. Unless stated otherwise, tensor shapes are logical shapes rather than a required physical memory layout.

Research anchors: [1], [3], [4], [5], [6], [7], [8], [10], [16], [17]

1 - WHAT A TRANSFORMER BLOCK IS

A repeated residual computation

A Transformer is built by stacking blocks that repeatedly update a sequence of hidden vectors. For a decoder-only language model, let

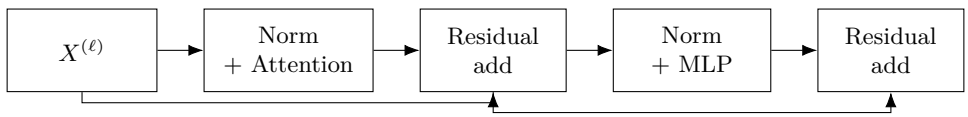
$$X^{(\ell)} \in \mathbb{R}^{B\times T\times d}$$

be the residual representation entering block ℓ. A common modern block has two main trainable sublayers: self-attention and a feed-forward network (MLP). In a Pre-Norm form,

$$U = X^{(\ell)} + \mathrm{Attn}(\mathrm{Norm}_1(X^{(\ell)})),$$

$$X^{(\ell+1)} = U + \mathrm{MLP}(\mathrm{Norm}_2(U)).$$

The attention branch can mix information across token positions. The MLP acts independently on each position but transforms its feature dimensions. Residual additions keep both sublayers connected to a shared d-dimensional stream.



Important. "Transformer block" is not one exact implementation. The durable skeleton is repeated residual updates around attention and feature-mixing sublayers. Normalization placement, attention type, activation, gating, and even whether the branches are serial can vary.

Research anchors: [1], [4], [8], [10]

2 - THE RESIDUAL STREAM

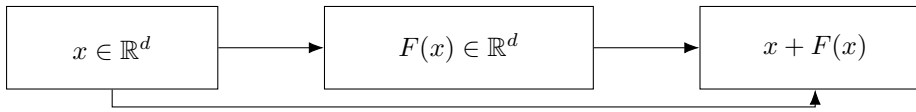
The state that every block reads and updates

The phrase *residual stream* is a useful name for the sequence of d-dimensional vectors carried through the stack by residual connections. If a sublayer computes an update $F(x)$, the residual rule is

$$y = x + F(x).$$

The identity path means the old representation remains directly available while the sublayer contributes a learned modification. This idea predates Transformers: residual networks were introduced to make very deep networks easier to optimize by learning residual functions relative to their inputs. [2]

For a Transformer block, attention and the MLP both write updates back into the same width-d representation. That requirement explains an architectural invariant that can otherwise look arbitrary: even when attention temporarily splits into heads or an MLP expands to a much wider hidden dimension, the branch must project back to d before the residual addition.



Mental model. A block does not replace the representation with "the attention output" and then "the MLP output." Each branch proposes an update to a shared residual representation.

Research anchors: [1], [2]

3 - SHAPES THROUGH THE BLOCK

Track the dimensions before tracking the equations

For a decoder block with residual width d,

$$X \in \mathbb{R}^{B\times T\times d}.$$

Normalization preserves this shape. Attention projects the last dimension into query, key, and value heads. With H_q query heads, H_{kv} KV heads, and head dimension d_h,

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \qquad K, V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

After attention, head outputs are combined and projected back to

$$A \in \mathbb{R}^{B\times T\times d}.$$

The MLP then expands each token representation to an intermediate width m and projects back:

$$B \times T \times d \to B \times T \times m \to B \times T \times d.$$

Stage	Logical shape	What changes?
Residual input	$B \times T \times d$	nothing
Norm	$B \times T \times d$	statistics/scale, not shape
Q	$B \times H_q \times T \times d_h$	split into query heads
K,V	$B \times H_{kv} \times T \times d_h$	KV heads may differ from Q
Attention output	$B \times T \times d$	heads recombined/projected
MLP hidden	$B \times T \times m$	feature expansion
Block output	$B \times T \times d$	back on residual stream

Important. The MLP width m and attention head dimensions are temporary internal widths. Residual addition requires the branch output to return to d.

Research anchors: [1], [11], [20]

4 - THE ORIGINAL 2017 POST-LN BLOCK

Residual first, normalization after

The original Transformer applies a residual connection around each sublayer and then normalizes the sum. Vaswani et al. write the sublayer output as [1]

$$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x)).$$

For an encoder block, the two sublayers are self-attention and the position-wise feed-forward network. The decoder inserts a third sublayer for encoder-decoder cross-attention.

For the two-sublayer case, a simplified Post-LN block is

$$u = \mathrm{LN}(x + \mathrm{Attn}(x)),$$

$$y = \mathrm{LN}(u + \mathrm{FFN}(u)).$$

This ordering matters. Normalization is not merely a box that can be moved without changing the function or optimization behavior.

The original model also used sinusoidal positional encodings added to embeddings and a ReLU feed-forward network. Those choices are historically important, but they are not part of the abstract definition of a Transformer block.

Mental model. Post-LN means the residual sum is normalized before it becomes the input to the next sublayer or block.

Research anchors: [1], [3]

5 - THE MODERN PRE-LN DECODER BLOCK

Normalize the branch input, keep the residual path direct

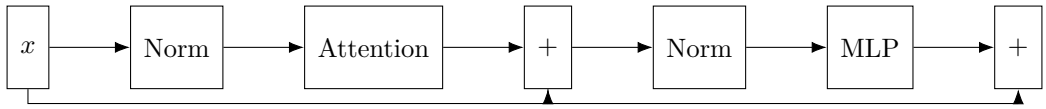
GPT-2 moved layer normalization to the input of each sub-block and added a final normalization after the last block. [4] GPT-3 retained GPT-2's pre-normalization architecture. [18] LLaMA later combined Pre-Norm with RMSNorm, SwiGLU, and RoPE. [8]

A common serial Pre-Norm block is

$$u = x + \mathrm{Attn}(\mathrm{Norm}_1(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}_2(u)).$$

A final model-level norm is often applied after the last block before the language-model output head, but that is a model architecture choice rather than part of every individual block.



Important. "Modern Transformer" does not imply one universal Pre-Norm recipe. Gemma 2, for example, uses both pre- and post-sublayer RMSNorm. [16]

Research anchors: [4], [8], [16], [18]

6 - WHY THE RESIDUAL ADDITION MATTERS

Identity path plus learned update

Consider a sublayer output

$$y = x + F(x).$$

The derivative contains an identity term:

$$\frac{\partial y}{\partial x} = I + \frac{\partial F(x)}{\partial x}.$$

This does not mean gradients can never vanish or explode in a deep Transformer. Normalization, initialization, parameter scale, optimizer dynamics, and the composition of many blocks still matter. But the direct identity path is the structural reason residual architectures can preserve and refine representations instead of forcing every layer to relearn them from scratch. [2]

Residual connections also impose shape discipline. If $x \in \mathbb{R}^d$, then $F(x)$ must be in $\mathbb{R}^d$ at the point of addition. That is why an attention branch with many heads ends in an output projection and why an MLP that expands to width m ends with a down projection.

Important. A residual connection is not concatenation. It is elementwise addition of same-shaped tensors. Concatenation would change the width and require different downstream parameters.

Research anchors: [2]

7 - LAYERNORM MECHANICS

Center, scale, then apply learned affine parameters

For one token vector $x \in \mathbb{R}^d$, LayerNorm computes statistics across the normalized feature dimensions. In the common one-dimensional case,

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

$$\mathrm{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

The original LayerNorm paper introduced per-neuron gain and bias after normalization. [3] Current PyTorch `LayerNorm` likewise supports learnable elementwise scale and bias by default. [19]

For sequence models, LayerNorm is typically applied independently to each token over its feature dimension, not across batch examples and not across sequence positions.

Mental model. LayerNorm changes the coordinate statistics of each token representation but leaves the tensor shape unchanged.

Research anchors: [3], [19]

8 - RMSNORM MECHANICS

Scale by root-mean-square without subtracting the mean

RMSNorm removes LayerNorm's explicit re-centering step. For $x \in \mathbb{R}^d$,

$$\mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{d}\sum_{i=1}^{d} x_i^2},$$

$$\mathrm{RMSNorm}(x)_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

Zhang and Sennrich proposed RMSNorm on the hypothesis that LayerNorm's re-centering invariance is dispensable in many settings. [6] Current PyTorch's RMSNorm follows the same core operation and preserves input shape. [20]

	LayerNorm	RMSNorm
Subtract mean	yes	no
Scale statistic	standard deviation	root mean square
Learned scale	usually yes	usually yes
Learned bias	commonly available	not part of core RMSNorm formula
Shape change	none	none

LLaMA uses RMSNorm in a Pre-Norm Transformer, and Qwen3 likewise reports RMSNorm with pre-normalization. [8,17]

Research anchors: [6], [8], [17], [20]

9 - WORKED NORM EXAMPLE

LayerNorm and RMSNorm do different things to the same vector

Ignore learned gain/bias and take

$$x = [1, 2, 3, 4].$$

For LayerNorm,

$$\mu = 2.5, \qquad \sigma^2 = 1.25, \qquad \sigma \approx 1.1180.$$

Ignoring ϵ for readability,

$$\mathrm{LN}(x) \approx [-1.3416, -0.4472, 0.4472, 1.3416].$$

The normalized vector is centered around zero.

For RMSNorm,

$$\mathrm{RMS}(x) = \sqrt{\frac{1^2 + 2^2 + 3^2 + 4^2}{4}} = \sqrt{7.5} \approx 2.7386,$$

so

$$\mathrm{RMSNorm}(x) \approx [0.3651, 0.7303, 1.0954, 1.4606].$$

Its mean is not forced to zero because the operation never subtracts μ.

Important. LayerNorm and RMSNorm are not interchangeable formulas with different names. A model checkpoint is trained with a specific normalization rule and parameterization.

Research anchors: [3], [6]

10 - PRE-LN VS POST-LN

Same ingredients, different optimization path

The algebraic distinction is compact:

$$\text{Post-LN:} \qquad y = \text{Norm}(x + F(x)),$$

$$\text{Pre-LN:} \qquad y = x + F(\text{Norm}(x)).$$

In Post-LN, the identity path passes through normalization before reaching the next block. In Pre-LN, the residual stream has a direct additive path across the sublayer.

Xiong et al. analyzed the two placements with mean-field theory. At initialization, they found large expected gradients near the output layers in the original Post-LN Transformer, motivating learning-rate warm-up, while the Pre-LN formulation had better-behaved gradients in their analysis and experiments. [5]

That result is evidence about optimization behavior, not a universal statement that Pre-LN is always better. Later architectures have revisited normalization and residual scaling in different ways. DeepNorm, for example, modifies residual connections and initialization to stabilize very deep Transformers. [21]

Mental model. Normalization placement changes the computational graph that gradients traverse. It is not a cosmetic rearrangement.

Research anchors: [1], [4], [5], [21]

11 - THE ATTENTION BRANCH

The block's token-mixing sublayer

For hidden states $X \in \mathbb{R}^{B\times T\times d}$, self-attention first forms learned projections. In a conventional multi-head formulation,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

After reshaping into heads, scaled dot-product attention computes

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

where M represents masking or attention bias. Head outputs are then recombined and projected through W_O back to width d before the residual addition. [1]

The attention branch is where token positions can exchange information. That makes it fundamentally different from the MLP branch, which applies the same feature transformation independently to each token position.

Important. The residual branch receives the attention *output projection*, not the attention-probability matrix. Attention weights are an intermediate.

Research anchors: [1], [22]

12 - WHERE ROPE ENTERS

Position modifies Q/K compatibility inside attention

Modern decoder-only LLMs often use Rotary Position Embedding (RoPE). RoPE does not alter the residual stream by adding a positional vector to X. Instead, standard RoPE applies position-dependent rotations to query and key components before their dot products are evaluated. [12]

In a Pre-Norm block, the conceptual order is therefore

$$X \to \text{Norm}(X) \to Q, K, V \to \text{RoPE}(Q, K) \to \text{Attention}.$$

LLaMA explicitly replaced absolute positional embeddings with RoPE at each layer. [8]

This distinction matters because it separates three objects that are often collapsed together:

- the residual representation X;
- the learned Q/K/V projections derived from X;
- the position-dependent transformation applied to Q and K.

Important. RoPE is not "added to the token embedding" in LLaMA-style blocks. It changes the Q/K interaction inside attention.

Research anchors: [8], [12]

13 - MHA, GQA, MQA, AND MLA INSIDE THE BLOCK

The block skeleton stays while the attention representation changes

The residual topology does not tell you how many key/value heads exist or what representation is cached during decoding.

With H_q query heads and H_{kv} key/value heads,

$$Q \in \mathbb{R}^{B\times H_q\times T\times d_h}, \quad K, V \in \mathbb{R}^{B\times H_{kv}\times T\times d_h}.$$

MHA typically uses $H_{kv} = H_q$. MQA uses one KV head. GQA uses an intermediate number. [11] Qwen3, for example, reports GQA across its dense models. [17]

MLA takes a different route: DeepSeek-V2 compresses key/value information into a lower-dimensional latent representation and handles RoPE-sensitive components separately. [14] Yet after its attention computation, the branch still returns a width-d update to the residual stream.

Mental model. MHA/GQA/MQA/MLA change the internals of the attention branch. They do not replace the surrounding norm-residual-MLP structure of the block.

Research anchors: [11], [14], [17]

14 - THE OUTPUT PROJECTION CLOSES THE ATTENTION BRANCH

Heads must return to the residual width

Suppose each of H_q query heads produces a vector of dimension d_h. Concatenating head outputs yields a representation of width $H_q d_h$. In the common case $H_q d_h = d$, the concatenated tensor already has width d, but it is still passed through a learned output projection

$$W_O \in \mathbb{R}^{d \times d}.$$

The branch then returns

$$A = \text{Concat}(\text{head}_1, \ldots, \text{head}_{H_q}) W_O$$

with shape $B \times T \times d$, making

$$x + A$$

well-defined. [1]

The output projection is not merely a reshape. It is a learned mixing of head outputs back into the model's residual feature space.

Important. Counting only W_Q, W_K, W_V misses one of the four major projection matrices in standard MHA: W_O.

Research anchors: [1]

15 - THE MLP IS POSITION-WISE

Feature mixing without token mixing

The original Transformer calls its second sublayer a position-wise feed-forward network because the same nonlinear function is applied independently to each sequence position. [1]

If $X \in \mathbb{R}^{B \times T \times d}$ and

$$f : \mathbb{R}^d \to \mathbb{R}^d,$$

then

$$Y_{b,t,:} = f(X_{b,t,:})$$

for every batch element b and token position t. The MLP does not directly read neighboring token positions. All cross-token information available to it must already be present in that token's current hidden state, typically after attention has mixed information across the sequence.

This gives the block a useful division of labor:

- attention mixes information *across positions*;
- the MLP transforms information *within each position's feature vector*.

Mental model. "Position-wise" means shared function, independent application across token positions. It does not mean the MLP is unaware of context: its input state can already contain context mixed in by earlier attention.

Research anchors: [1]

16 - THE ORIGINAL RELU FEED-FORWARD NETWORK

Expand, activate, project back

The 2017 Transformer uses two linear transformations with a ReLU between them: [1]

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

With residual width d and feed-forward width m,

$$W_1 \in \mathbb{R}^{d\times m}, \qquad W_2 \in \mathbb{R}^{m\times d}.$$

The original base Transformer used $d = 512$ and $m = 2048$, a $4\times$ expansion.

Ignoring biases, the parameter count of this dense FFN is

$$P_{\mathrm{FFN}} = 2dm.$$

For $m = 4d$,

$$P_{\mathrm{FFN}} = 8d^2.$$

That is why the feed-forward branch can contain a large fraction of a dense Transformer's parameters even though it never mixes token positions.

Important. The $4d$ ratio is a design choice from the original architecture, not a mathematical requirement of Transformers. Modern gated MLPs often use different expansion ratios.

Research anchors: [1]

17 - FROM GELU TO GATED MLPS

The nonlinearity is part of the architecture

Transformer MLPs did not remain fixed at ReLU. Later Transformer language models adopted GELU-family activations, while subsequent architectures also adopted gated linear-unit variants. A generic gated unit uses two learned projections and combines them elementwise:

$$\mathrm{Gate}(x) = \phi(xW_g) \odot (xW_u).$$

A third projection maps the gated hidden representation back to width d.

Shazeer systematically compared GLU variants in Transformer feed-forward sublayers and found several gated variants, including SwiGLU and GEGLU, improved quality over the commonly used ReLU or GELU baselines in the studied settings. [7]

The activation choice changes more than one function call. Gating introduces an additional projection matrix and therefore changes parameter accounting, compute, and the hidden-width choice used for compute-equivalent comparisons.

Research anchors: [7], [23]

18 - SWIGLU MECHANICS

Two upward projections, one gated product, one downward projection

A common SwiGLU MLP can be written

$$g = \mathrm{SiLU}(xW_g), \qquad u = xW_u,$$

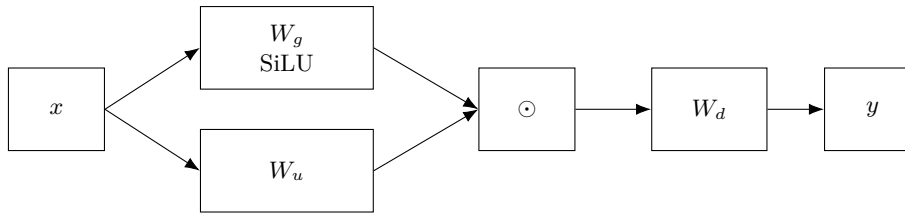
$$h = g \odot u, \qquad y = hW_d,$$

where $W_g, W_u \in \mathbb{R}^{d\times m}$ and $W_d \in \mathbb{R}^{m\times d}$. SiLU (also called Swish with $\beta = 1$) is

$$\mathrm{SiLU}(z) = z\,\sigma(z).$$

The elementwise product gates one projected stream by another transformed stream.

LLaMA explicitly replaced ReLU with SwiGLU and reduced the hidden width relative to a $4d$ FFN to account for the third matrix. [8] PaLM likewise used SwiGLU and emphasized compute-equivalent comparisons. [10]



Research anchors: [7], [8], [10]

19 - WHY GATED MLPS USE A DIFFERENT WIDTH

Match parameter budget before comparing activations

A two-matrix FFN has approximately

$$P_{\text{FFN}} = 2dm.$$

A three-matrix SwiGLU-style FFN has approximately

$$P_{\text{SwiGLU}} = 3dm.$$

If the ReLU baseline uses $m_{\text{ReLU}} = 4d$, then matching parameter count gives

$$2d(4d) = 3d\, m_{\text{SwiGLU}},$$

so

$$m_{\text{SwiGLU}} = \frac{8}{3}d \approx 2.67d.$$

This is the reason it is misleading to compare a $4d$ ReLU FFN against a $4d$ SwiGLU FFN and attribute every difference to the activation: the latter has an extra large projection matrix.

LLaMA describes its SwiGLU dimension as $\frac{2}{3}(4d)$ before architecture-specific rounding. [8]

Mental model. When the number of projection matrices changes, the hidden width must be part of any fair parameter/compute comparison.

Research anchors: [7], [8], [10]

20 - WORKED PARAMETER EXAMPLE

Why the MLP often dominates a dense block's parameters

Take an illustrative block with

$$d = 768, \qquad H_q = H_{kv} = 12, \qquad d_h = 64.$$

For standard MHA, ignoring biases,

$$P_{\text{attn}} = 4d^2 = 4(768)^2 = 2{,}359{,}296,$$

covering W_Q, W_K, W_V, W_O.

A ReLU FFN with $m = 4d = 3072$ has

$$P_{\text{FFN}} = 2(768)(3072) = 4{,}718{,}592.$$

So the two major branches contain about

$$7{,}077{,}888$$

projection parameters before norms and biases, with the FFN accounting for two-thirds of that total.

Now choose a parameter-matched SwiGLU width $m = 2048 = \frac{8}{3}d$:

$$P_{\text{SwiGLU}} = 3(768)(2048) = 4{,}718{,}592.$$

The gated MLP has exactly the same matrix-parameter count as the $4d$ two-matrix FFN in this example.

Important. This is an illustrative dense MHA block. GQA/MQA reduce K/V projection parameters, and MoE changes total vs active parameter accounting.

Research anchors: [1], [7], [8], [11]

21 - WORKED SHAPE TRACE

One block, end to end

Take

$$B = 2, \quad T = 4, \quad d = 8, \quad H_q = H_{kv} = 2, \quad d_h = 4, \quad m = 32.$$

The residual input is

$$X : [2, 4, 8].$$

Normalization keeps $[2, 4, 8]$. Q/K/V reshape to

$$[2, 2, 4, 4].$$

For self-attention, the per-head score tensor has shape

$$[2, 2, 4, 4],$$

because each of 4 query positions scores 4 key positions. After value aggregation and head concatenation, the attention branch returns

$$[2, 4, 8].$$

The first residual add therefore remains $[2, 4, 8]$.

The MLP expands each token independently:

$$[2, 4, 8] \to [2, 4, 32] \to [2, 4, 8].$$

The second residual add again leaves the block output at

$$[2, 4, 8].$$

Mental model. The sequence length stays T through an ordinary block. Attention changes how positions interact, while residual width returns to d after every branch.

Research anchors: [1]

22 - SERIAL VS PARALLEL RESIDUAL BLOCKS

Attention and MLP need not be strictly sequential

The familiar Pre-Norm block is serial: the MLP receives the residual state after the attention update. In simplified form,

$$u = x + \mathrm{Attn}(\mathrm{Norm}(x)),$$

$$y = u + \mathrm{MLP}(\mathrm{Norm}(u)).$$

PaLM used a parallel formulation in which attention and the MLP read the same normalized residual input and their outputs are added together: [10]

$$y = x + \mathrm{Attn}(\mathrm{Norm}(x)) + \mathrm{MLP}(\mathrm{Norm}(x)).$$

The PaLM paper reports that this enabled fusion of the attention and MLP input matrix multiplications and produced roughly 15% faster training at large scale in their setup, while their ablations showed scale-dependent quality effects. [10]

Important. Parallel residual blocks are not algebraically identical to serial blocks. The MLP no longer sees the current block's attention update before producing its own update.

Research anchors: [10]

23 - ENCODER AND DECODER BLOCK FAMILIES

The sublayers depend on the model family

The 2017 Transformer uses two different stacks. [1]

Block family	Self-attention	Cross-attention	MLP
Encoder	bidirectional/full	no	yes
Decoder-only LM	causal	no	yes
Encoder-decoder decoder	causal	yes, over encoder output	yes

Cross-attention adds another residual sublayer because its queries come from the decoder state while its keys/values come from encoder representations. A decoder-only LLM omits that cross-attention sublayer entirely.

Modern multimodal systems can reintroduce cross-attention or other modality-specific blocks, but "Transformer block" should always be interpreted relative to the architecture being discussed.

Mental model. Self-attention describes where Q/K/V come from. Causality describes which positions can interact. Cross-attention describes a separate source for K/V.

Research anchors: [1]

24 - CAUSAL MASKING IS INSIDE ATTENTION

Block topology and connectivity are different questions

A decoder-only block is often called a "causal Transformer block," but causality is enforced inside the attention computation. For query position i and key position j, an additive mask can use

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Then attention computes

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right) V.$$

The surrounding residual/MLP topology does not become causal by itself. The MLP is position-wise, so it does not create cross-position leakage, but the attention sublayer must enforce the permitted connectivity.

Current PyTorch scaled-dot-product attention exposes causal behavior as an attention operation and documents separate query and source lengths for non-square cases. [22]

Important. "Decoder block" is not synonymous with "lower-triangular matrix everywhere." The causal restriction belongs to the attention relation between query and key positions.

Research anchors: [1], [22]

25 - MOE REPLACES THE MLP BRANCH

Sparse experts change conditional computation, not the block skeleton

In many sparse Mixture-of-Experts Transformers, the dense MLP sublayer is replaced by a routed expert bank. For token state x, a router selects a small set of experts and combines their outputs:

$$y_{\text{MoE}} = \sum_{i \in \text{Top-}k} \tilde{p}_i(x) E_i(x).$$

The MoE branch still must return a vector of width d so the residual update remains valid:

$$x \leftarrow x + \text{MoE}(\text{Norm}(x)).$$

This is why MoE can be understood as a change to the feed-forward branch rather than a wholly different Transformer topology.

Qwen3's MoE models, for example, share the same fundamental dense-model architecture around attention/norms but replace the feed-forward computation with fine-grained routed experts. [17]

Important. MoE sparsity and attention sparsity are different. MoE sparsifies which feed-forward parameters are active for a token; sparse attention changes which token pairs interact.

Research anchors: [17], [24]

26 - QK-NORM IS NOT BLOCK NORMALIZATION

Normalization can appear inside the attention branch too

Pre-Norm or Post-LN describes where a norm sits relative to the residual branch. Modern models can also normalize internal attention tensors.

Qwen3 reports RMSNorm with pre-normalization at the block level and, separately, QK-Norm in the attention mechanism to stabilize training. [17] These are different normalization sites:

- **block norm**: transforms the residual-stream input before or after a sublayer;
- **QK-Norm**: normalizes query/key representations inside attention before score computation.

Moving or removing one cannot be reasoned about as if it were the other.

Mental model. A model can contain several normalizations with different tensors, axes, purposes, and parameters. "Uses RMSNorm" is not enough to reconstruct the block.

Research anchors: [17]

27 - GEMMA 2: A USEFUL COUNTEREXAMPLE

Modern blocks do not all converge to one norm placement

It is tempting to summarize contemporary LLM blocks as "Pre-RMSNorm + attention + SwiGLU." Gemma 2 is a useful reason not to universalize that pattern.

Its technical report specifies both pre-norm and post-norm with RMSNorm: the input and output of each attention and feed-forward sublayer are normalized. It also uses GeGLU rather than SwiGLU, GQA, and alternating local/global attention layers. [16]

That means a Gemma 2-style sublayer is closer conceptually to

$$y = x + \mathrm{PostNorm}(F(\mathrm{PreNorm}(x))),$$

with exact implementation details determined by the model.

Important. The word "Transformer" identifies a family, not a frozen 2017 wiring diagram. Accurate explanations should state which architecture is being described.

Research anchors: [16]

28 - TRAINING, PREFILL, AND DECODE

Same block parameters, very different execution shapes

The model does not switch to a different mathematical block when it begins generating tokens. What changes is the shape of the work and what state can be reused.

	Training/full sequence	Prompt prefill	Incremental decode
Query positions	many	many	usually one/few new positions
Past K/V	computed in pass	build cache	read from cache
MLP positions	many in parallel	many in parallel	new position(s)
Sequence dependency	causal mask for LM	causal mask	history already cached

During decode, the newest token still passes through every Transformer block. At each attention sublayer, its query attends to cached K/V history. The MLP remains position-wise and only needs to process the new token state for that decode step.

Mental model. KV caching changes how the attention branch reuses history. It does not skip the rest of the block for the newly generated token.

Research anchors: [4], [22]

29 - THE KV CACHE BELONGS TO ATTENTION STATE

What is persistent across decode steps

For a standard causal block, the reusable per-token history is the attention key/value state (or an architecture-specific compressed equivalent), not the output of the entire block as a generic "memory."

At decode step t, a layer computes a new query/key/value for the new position. The new query interacts with cached keys and values from earlier positions. The new K/V are appended for use by later steps. The MLP has no analogous sequence cache because it is position-wise: once the current token leaves the MLP, that branch has no need to reread its earlier intermediate activations during future decode steps.

Architectures such as MLA change exactly what is cached, but the reusable state is still part of the attention mechanism. [14]

Important. KV cache memory grows with cached sequence state. Residual-stream width and MLP width are separate resource questions.

Research anchors: [14], [22]

30 - BLOCK-LEVEL PARAMETER ACCOUNTING

Attention and MLP formulas expose architecture trade-offs

Ignoring biases and norms, standard MHA with $H_q d_h = d$ uses approximately

$$P_{\text{MHA}} = 4d^2.$$

With GQA, the Q and output matrices remain roughly d^2 each, while K and V project only to $H_{kv} d_h$:

$$P_{\text{GQA}} \approx 2d^2 + 2d(H_{kv} d_h).$$

For $d = 4096$, $H_q = 32$, $H_{kv} = 8$, $d_h = 128$:

$$P_{\text{MHA}} = 67{,}108{,}864,$$

$$P_{\text{GQA}} = 41{,}943{,}040.$$

That is a 37.5% reduction in these attention-projection parameters for this illustrative configuration.

A dense two-matrix $4d$ FFN would contain $8d^2$ matrix parameters, while a parameter-matched SwiGLU is also near $8d^2$. These formulas explain why dense LLM blocks often devote more matrix parameters to the MLP than to attention.

Important. Parameter count is not runtime. Sequence length, kernel efficiency, KV traffic, batching, and hardware can make attention or the MLP dominate different execution regimes.

Research anchors: [1], [7], [11]

31 - COMPUTE INTUITION: ATTENTION VS MLP

Sequence mixing and feature mixing scale differently

For dense self-attention over sequence length T, the QK and AV products contain a quadratic term in sequence length, roughly

$$O(T^2 d)$$

when head dimensions are aggregated. Linear projections contribute terms roughly proportional to Td^2.

A dense MLP applies independently to each position, so its dominant matrix multiplications scale roughly as

$$O(Tdm)$$

for a two-matrix FFN (with a constant-factor change for gated variants).

This means the relative bottleneck depends on the shape regime. At modest context length and large model width, MLP/projection GEMMs can be a major share of work. At long dense contexts, attention's pairwise sequence term grows rapidly. During one-token decoding, attention faces growing KV reads while the MLP continues processing only the current token.

Mental model. Do not ask "is attention or the MLP more expensive?" without specifying sequence length, batch, width, architecture, phase, precision, and hardware.

Research anchors: [1], [22]

32 - ACTIVATION MEMORY AND RECOMPUTATION

Training stores more than parameters

During training, backpropagation needs information from the forward pass. Transformer blocks can therefore consume substantial activation memory in addition to weight, gradient, and optimizer-state memory.

A naive implementation may retain normalized inputs, Q/K/V projections, attention intermediates, MLP activations, and residual states needed by backward kernels. Fused kernels and recomputation strategies can change exactly what is stored.

Activation checkpointing deliberately saves fewer intermediate activations and recomputes parts of the forward pass during backward, trading extra computation for lower activation memory. That is a training-system strategy around the block, not a change to the mathematical function the block represents.

Important. "Block memory" is ambiguous. Parameter memory, training activations, optimizer state, temporary kernel workspace, and inference KV cache are different quantities.

Research anchors: [25]

33 - DROPOUT, BIASES, AND OTHER SMALL CHOICES

Configuration details are not universal Transformer laws

Two implementations can share the same high-level block diagram while differing in many details:

- whether linear projections include bias terms;
- where dropout is applied and whether it is zero in a given model;
- the normalization epsilon;
- whether embeddings/output weights are tied;
- exact rotary dimensions and scaling;
- whether attention uses local windows, soft-capping, or extra normalization;
- activation and gated-MLP convention;
- tensor parallel or fused-kernel layout.

These choices can affect checkpoint compatibility, parameter counts, numerical behavior, and kernel selection. They should be read from the model's paper/configuration/code rather than inferred from the word "Transformer."

Gemma 2's logit soft-capping and Qwen3's QK-Norm are examples of architecture details that are easy to omit from a generic diagram but matter to an exact implementation. [16,17]

Research anchors: [16], [17]

34 - A TRANSPARENT PYTORCH BLOCK

Pedagogical Pre-RMSNorm attention + SwiGLU

The following executable sketch exposes the order of operations. It uses ordinary MHA and intentionally omits production concerns such as GQA, RoPE, KV caching, fused kernels, tensor parallelism, dropout, and model-specific biases.

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        assert d % n_heads == 0
        self.h, self.dh = n_heads, d // n_heads
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.out = nn.Linear(d, d, bias=False)

    def forward(self, x):
        B, T, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        def heads(z):
            return z.view(B, T, self.h, self.dh).transpose(1, 2)
        q, k, v = map(heads, (q, k, v))
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        y = y.transpose(1, 2).reshape(B, T, D)
        return self.out(y)

class Block(nn.Module):
    def __init__(self, d, n_heads, mlp_dim):
        super().__init__()
        self.norm1, self.norm2 = nn.RMSNorm(d), nn.RMSNorm(d)
        self.attn = CausalSelfAttention(d, n_heads)
        self.gate = nn.Linear(d, mlp_dim, bias=False)
        self.up = nn.Linear(d, mlp_dim, bias=False)
        self.down = nn.Linear(mlp_dim, d, bias=False)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        h = self.norm2(x)
        h = F.silu(self.gate(h)) * self.up(h)
        return x + self.down(h)
```

This matches the serial Pre-Norm residual topology and the three-projection SwiGLU structure. A real model must match its exact attention head layout, RoPE rule, norm epsilon, MLP dimensions, cache semantics, and checkpoint parameter names.

Important. Pedagogical code is not a drop-in reproduction of LLaMA, Qwen, Gemma, or another model. Exact architecture is a compatibility contract.

Research anchors: [6], [7], [8], [17], [20], [22]

35 - COMMON IMPLEMENTATION MISTAKES

Most block bugs are ordering, shape, or contract bugs

1. **Using Post-LN equations for a Pre-LN checkpoint.** Same modules, different function.
2. **Normalizing the wrong tensor.** Block norm, QK-Norm, and post-sublayer norm are different sites.
3. **Forgetting the attention output projection.** Head concatenation is not the full branch.
4. **Applying RoPE to the residual stream or V by default.** Standard LLaMA-style RoPE targets Q/K.
5. **Assuming $H_q = H_{kv}$.** GQA/MQA break that equality.
6. **Using a two-matrix FFN parameter formula for SwiGLU.** Gating adds a third large projection.
7. **Adding tensors with mismatched widths.** Every residual branch must return to d.
8. **Treating causal masking as a block-level operation.** It belongs inside attention connectivity.
9. **Copying a generic MLP width.** Expansion ratios are architecture-specific.
10. **Equating a working tensor shape with checkpoint compatibility.** Epsilon, bias, tying, ordering, and conventions still matter.

Research anchors: [1], [8], [11], [12], [16], [17]

36 - COMMON MISCONCEPTIONS

Shortcuts that create the wrong mental model

Misconception	Correction
"The attention output replaces the token state."	It is usually added as a residual update.
"The MLP mixes tokens."	It acts position-wise; context can already be present in each state.
"RMSNorm is LayerNorm without bias."	RMSNorm also omits mean subtraction and uses a different scale statistic.
"Pre-LN and Post-LN are equivalent reorderings."	They define different computation/gradient paths.
"RoPE modifies token embeddings."	Standard RoPE rotates Q/K inside attention.
"Every modern LLM is Pre-RMSNorm + SwiGLU."	Gemma 2, PaLM, Qwen3, and others illustrate important variants.
"GQA changes the whole block."	It changes attention's Q/K/V head structure.
"MoE means a different Transformer skeleton."	It commonly replaces the feed-forward branch with routed experts.
"The KV cache stores the whole block state."	It stores attention K/V or an architecture-specific equivalent.
"Attention always dominates compute."	The bottleneck depends on shape, phase, architecture, and hardware.

Research anchors: [1], [6], [8], [10], [11], [14], [16], [17]

37 - THE COMPLETE MENTAL MODEL

The Transformer block in one chain

A modern Transformer block is easiest to remember as nine linked ideas.

1. **Carry a residual state.** Each token has a width-d vector that persists through the stack.
2. **Normalize at defined sites.** LayerNorm/RMSNorm placement is part of the architecture.
3. **Mix tokens with attention.** Q/K/V projections, position treatment, masking, and head layout determine how positions interact.
4. **Project back to d.** The attention branch must return to residual width.
5. **Add the update.** Residual addition preserves the old representation while incorporating attention's contribution.
6. **Transform features with the MLP.** Each token is expanded, nonlinearly gated/activated, and projected back independently.
7. **Add again.** The MLP contributes another residual update.
8. **Repeat with architecture-specific variants.** GQA, QK-Norm, GeGLU/SwiGLU, local attention, post-norm, MoE, and other choices alter internals without erasing the core logic.
9. **Interpret execution by phase.** Training, prefill, and decode run the same learned block under different shapes and reuse patterns.

Mental model. The durable abstraction is not "attention followed by an MLP." It is a residual state repeatedly updated by token-mixing and feature-mixing computations whose normalization, routing, and attention details are architecture-specific.

Research anchors: [1], [4], [6], [7], [8], [10], [16], [17]

38 - A COMPACT ARCHITECTURE COMPARISON

Representative blocks, not a universal progression

Model/family	Norm placement	MLP	Position/attention	Useful lesson
Transformer (2017)	Post-LN	ReLU, $4d$	sinusoidal + MHA	original reference block
GPT-2	Pre-LN + final norm	GELU-style	causal MHA	pre-normalization moved into sub-block input
PaLM	Pre-LN, parallel branches	SwiGLU	MQA	serial ordering is not required
LLaMA	Pre-RMSNorm	SwiGLU	RoPE + MHA	common modern dense recipe
Gemma 2	pre + post RMSNorm	GeGLU	GQA, local/global	modern norm placement still varies
Qwen3	Pre-RMSNorm + QK-Norm	SwiGLU	RoPE + GQA	internal attention norms can coexist with block norm

This table is intentionally selective. It shows why the phrase "Transformer block" should trigger a checklist rather than a single memorized drawing: norm placement, branch ordering, attention head structure, positional method, MLP gating, and internal attention modifications can all change.

Important. Model families evolve across versions and parameter scales. For exact reproduction, use the corresponding technical report and released configuration rather than this comparison table.

Research anchors: [1], [4], [8], [10], [16], [17]

39 - REFERENCES AND FURTHER READING

1. A. Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762.
2. K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016. arXiv:1512.03385.
3. J. L. Ba, J. R. Kiros, G. E. Hinton. *Layer Normalization.* arXiv:1607.06450, 2016.
4. A. Radford et al. *Language Models are Unsupervised Multitask Learners.* OpenAI technical report, 2019.
5. R. Xiong et al. *On Layer Normalization in the Transformer Architecture.* ICML 2020. arXiv:2002.04745.
6. B. Zhang, R. Sennrich. *Root Mean Square Layer Normalization.* NeurIPS 2019. arXiv:1910.07467.
7. N. Shazeer. *GLU Variants Improve Transformer.* arXiv:2002.05202, 2020.
8. H. Touvron et al. *LLaMA: Open and Efficient Foundation Language Models.* arXiv:2302.13971, 2023.
9. H. Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models.* arXiv:2307.09288, 2023.
10. A. Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways.* JMLR 24, 2023.
11. J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* EMNLP 2023. arXiv:2305.13245.
12. J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864, 2021.
13. T. Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135.
14. DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434, 2024.
15. A. Jiang et al. *Mixtral of Experts.* arXiv:2401.04088, 2024.
16. Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size.* arXiv:2408.00118, 2024.
17. A. Yang et al. *Qwen3 Technical Report.* arXiv:2505.09388, 2025.
18. T. Brown et al. *Language Models are Few-Shot Learners.* NeurIPS 2020. arXiv:2005.14165.
19. PyTorch documentation. `torch.nn.LayerNorm`. Current documentation.
20. PyTorch documentation. `torch.nn.RMSNorm`. Current documentation.
21. H. Wang et al. *DeepNet: Scaling Transformers to 1,000 Layers.* arXiv:2203.00555, 2022.
22. PyTorch documentation. `torch.nn.functional.scaled_dot_product_attention`. Current documentation.
23. D. Hendrycks, K. Gimpel. *Gaussian Error Linear Units (GELUs).* arXiv:1606.08415, 2016.
24. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* ICLR 2017. arXiv:1701.06538.
25. T. Chen et al. *Training Deep Nets with Sublinear Memory Cost.* arXiv:1604.06174, 2016.